

Master Time Complexity and Big O Notation in 30 Minutes

Introduction

Time Complexity is one of the most critical concepts in Data Structures and Algorithms (DSA). It determines which algorithm to use based on the input size. While many teachers assume students already know this, understanding time complexity deeply will transform your problem-solving approach in interviews and competitive programming.

Why Time Complexity Matters:

- Helps you choose the right algorithm for different input sizes
 - Prevents "Time Limit Exceeded" errors on platforms like LeetCode and GeeksforGeeks
 - Essential for understanding DSA patterns
 - Critical for coding interviews
-

Part 1: Fundamental Concepts

What is a Simple Operation?

A **simple operation** is any basic computational task that takes a constant amount of time to execute, regardless of the input size.

Examples of Simple Operations:

- `a = 10` (variable assignment)
- `print("Hello")` (print statement)
- `a = 10 + 2` (arithmetic operation)
- `a = b + c` (addition of two variables)
- `return a + b` (return statement)

Key Point: Each simple operation takes **1 unit of time** to execute. This unit could be 1 millisecond, 1 microsecond, or 1 second—the specific time doesn't matter; what matters is that it's constant.

Input Independent vs Input Dependent Operations

Input Independent Operations (Constant Time - $O(1)$):

- Operations whose execution time doesn't change with input size

- Example: `print("Hello")` takes the same time whether you have 1 element or 1 million elements in your array
- These operations take constant time: **$O(1)$**

Input Dependent Operations:

- Operations whose execution time changes based on the input size
 - Example: A loop that iterates n times where n is the input size
 - As input increases, execution time increases proportionally
-

Part 2: The 10^8 Rule

Understanding the LeetCode Constraint

When you submit a solution on platforms like LeetCode or GeeksforGeeks, there's a **Time Limit Exceeded (TLE)** error you might encounter. This happens because:

General Rule: Most online judges allow approximately **10^8 (100 million) simple operations** to execute within the time limit (typically 1 second).

Why 10^8 ?

- This limit is language-dependent:
 - C++: Can execute 10^8 operations in ~1 second
 - Python: Can execute 10^8 operations in ~2 seconds
 - JavaScript: Can execute 10^8 operations in ~3 seconds
 - Java: Can execute 10^8 operations in ~2 seconds
- Online judges adjust the time limit based on your chosen language
- The **number of operations remains constant**, but the time taken varies

How to Use This Rule

When you see an input constraint, you can immediately determine which algorithm's time complexity will work:

Given: Input size limit = n **Allowed operations:** 10^8

You can work backwards:

- If $n = 10^8 \rightarrow$ Maximum allowed complexity: **$O(n)$** (10^8 operations)
 - If $n = 10^5 \rightarrow$ Maximum allowed complexity: **$O(n^2)$** (10^{10} operations? No! This exceeds 10^8 , so not allowed)
 - If $n = 10^5 \rightarrow$ You can use **$O(n \log n)$** (roughly $10^5 \times 17 \approx 1.7 \times 10^6$, which is less than 10^8)
 - If $n = 10^3 \rightarrow$ Maximum allowed complexity: **$O(n^3)$** (10^9 operations? No! This exceeds 10^8)
-

Part 3: Input Size and Algorithm Selection

Examples with Different Input Constraints

Example 1: $n = 10$

- $O(n) \rightarrow 10$ operations
- $O(n^2) \rightarrow 100$ operations
- $O(n^3) \rightarrow 1,000$ operations
- $O(2^n) \rightarrow 1,024$ operations
- $O(n!) \rightarrow$ Much less than 10^8 operations
- $O(n^n) \rightarrow 10^{10}$ operations (exceeds 10^8 limit)

Example 2: $n = 1,000$

- $O(n) \rightarrow 1,000$ operations
- $O(n^2) \rightarrow 10^6$ operations
- $O(n^3) \rightarrow 10^9$ operations (exceeds 10^8 limit)

Example 3: $n = 10^5$

- $O(n) \rightarrow 10^5$ operations
- $O(n^2) \rightarrow 10^{10}$ operations (exceeds 10^8 limit)
- $O(n \log n) \rightarrow 10^5 \times 17 \approx 1.7 \times 10^6$ operations (works!)

Example 4: $n = 10^8$

- $O(n) \rightarrow 10^8$ operations
- $O(n \log n) \rightarrow 10^8 \times 27 \approx 2.7 \times 10^9$ operations (exceeds 10^8 limit)

Example 5: $n = 10^9$

- $O(n) \rightarrow 10^9$ operations (exceeds 10^8 limit)
 - $O(\log n) \rightarrow \log(10^9) \approx 30$ operations (works!)
-

Part 4: Understanding Big O Notation

$O(1)$ - Constant Time

Definition: Complexity $O(1)$ means the operation takes **constant time** that is **input-independent**.

Critical Understanding:

- $O(1)$ does NOT mean the operation takes exactly 1 unit of time
- $O(1)$ means the time is **constant regardless of input size**

- Whether input is 1, 100, or 1 million, the operation takes the same time

Example:

```
def get_first_element(arr):  
    return arr[0] # O(1) - Always takes same time
```

Real-world analogy:

- Getting the first item from a shelf takes the same time whether the shelf has 1 item or 1 million items
- The time is constant

$O(n)$ - Linear Time

Definition: Complexity $O(n)$ means the operation's execution time is **directly proportional to input size n** .

When input increases, time increases linearly:

- $n = 10 \rightarrow 10$ operations
- $n = 100 \rightarrow 100$ operations
- $n = 1,000 \rightarrow 1,000$ operations

Example:

```
def print_all_elements(arr): # n = size of array  
    for i in range(n):  
        print(arr[i]) # O(n) - Loops n times
```

Analysis:

- The loop runs n times
- Each iteration: $O(1)$ operation (print)
- Total: $n \times O(1) = O(n)$

$O(n^2)$ - Quadratic Time

Definition: Complexity $O(n^2)$ appears when you have **nested loops** where both loops depend on the same input size.

Example:

```
def print_all_pairs(arr): # n = size of array  
    for i in range(n):      # Outer loop: n times  
        for j in range(n):  # Inner loop: n times  
            print(arr[i], arr[j]) # O(1) operation
```

Analysis:

- Outer loop runs n times
- For each iteration of outer loop, inner loop runs n times
- For each iteration of inner loop: $O(1)$ operation
- Total: $n \times n \times O(1) = O(n^2)$

Visualization with $n = 3$:

```
i=0: j=0,1,2 (3 operations)
i=1: j=0,1,2 (3 operations)
i=2: j=0,1,2 (3 operations)
Total:  $9 = 3^2 = n^2$ 
```

$O(n^3)$ - Cubic Time

Definition: Three nested loops depending on the same input size.

Example:

```
for i in range(n):
    for j in range(n):
        for k in range(n):
            print(i, j, k) #  $O(1)$ 
```

Total: $n \times n \times n = O(n^3)$

$O(\log n)$ - Logarithmic Time

Definition: Complexity $O(\log n)$ appears when you **divide the problem size by 2 in each step** (binary search pattern).

Examples:

- Binary search
- Operations that repeatedly halve the input

Comparison:

- If $n = 10$, $\log_2(10) \approx 3.3 \rightarrow$ roughly 4 operations
- If $n = 1,000$, $\log_2(1,000) \approx 10 \rightarrow$ roughly 10 operations
- If $n = 1,000,000$, $\log_2(1,000,000) \approx 20 \rightarrow$ roughly 20 operations

$O(n \log n)$ - Linear Logarithmic Time

Definition: Complexity appears when you have an $O(n)$ loop but inside it, you're doing $O(\log n)$ work.

Common in:

- Merge Sort
- Quick Sort (average case)
- Most efficient comparison-based sorting algorithms

Example:

- $n = 10^5$
- Operations: $10^5 \times \log_2(10^5) \approx 10^5 \times 17 \approx 1.7 \times 10^6$ (which is less than 10^8 , so it passes!)

$O(2^n)$ - Exponential Time

Definition: Complexity doubles with each increase in input size. Usually appears in **recursive problems without memoization**.

Example:

- $n = 10 \rightarrow 2^{10} = 1,024$ operations
- $n = 20 \rightarrow 2^{20} = 1,048,576$ operations
- $n = 30 \rightarrow 2^{30} \approx 1$ billion operations (exceeds 10^8 limit!)

Problem: This complexity is generally acceptable only for very small input sizes ($n \leq 20$).

$O(n!)$ - Factorial Time

Definition: Extremely slow. Only acceptable for very small inputs ($n \leq 10$).

Example:

- $n = 10 \rightarrow 10! = 3,628,800$ operations
- $n = 12 \rightarrow 12! = 479,001,600$ operations

Part 5: Important Rules for Time Complexity Analysis

Rule 1: Identify Independent Sections

When analyzing a program:

1. Break the program into independent sections
2. Independent = one section doesn't depend on another
3. Each independent section can start and end without affecting others

Example:

```
print("Hello")           # Section 1: O(1)

for i in range(n):       # Section 2: O(n)
    print(i)

arr.sort()               # Section 3: O(n log n)
```

Rule 2: Sum Independent Sections, Take Maximum

When you have multiple independent sections:

Total Complexity = Maximum complexity among all sections

Why Maximum, not Sum?

Consider:

- Section 1: $O(1)$
- Section 2: $O(1)$
- Section 3: $O(1)$

Total: $O(1) + O(1) + O(1) = ?$

If we write this mathematically: $n^0 + n^0 + n^0 = 3 \times n^0$

The **highest power of n is 0**, so complexity = $O(n^0) = O(1)$

Another Example:

- Section 1: $O(n)$
- Section 2: $O(n^2)$

Total: $O(n) + O(n^2) = n + n^2$

The **highest power of n is 2**, so complexity = $O(n^2)$

Rule 3: Only the Highest Power Matters

When analyzing expressions:

- $n + 5 \rightarrow$ Highest power: 1 $\rightarrow O(n)$
- $n^2 + n + 5 \rightarrow$ Highest power: 2 $\rightarrow O(n^2)$
- $n^3 + 100n^2 + 50n \rightarrow$ Highest power: 3 $\rightarrow O(n^3)$
- $n^2 + 100 \rightarrow$ Highest power: 2 $\rightarrow O(n^2)$

Why? Because as n gets larger, the highest power term dominates all others.

Rule 4: Ignore Constant Multipliers

These all have the same complexity:

- $O(n)$, $O(2n)$, $O(100n)$, $O(n - 1)$, $O(n + 50) \rightarrow O(n)$
- $O(n^2)$, $O(5n^2)$, $O(n^2 - 1) \rightarrow O(n^2)$

Why? Constant differences become irrelevant for large inputs.

Part 6: Step-by-Step Examples

Example 1: Simple Function

```
def simple_function(arr):  
    print("Hello")      # O(1)  
    a = 10               # O(1)  
    b = 20               # O(1)  
    return a + b         # O(1)
```

Analysis:

- 4 independent statements, each $O(1)$
- Total: $\text{Max}(O(1), O(1), O(1), O(1)) = O(1)$

Example 2: Single Loop

```
def single_loop(arr): # arr has n elements  
    for i in range(n):  
        print("Hello") # O(1) operation
```

Analysis:

- Loop runs n times
- Each iteration: $O(1)$ operation
- Total: $n \times O(1) = O(n)$

Example 3: Loop with Specific Iteration Count

```
for i in range(100):  
    print("Hello")
```


Analysis:

- Loop runs exactly 100 times
- This is constant regardless of input n
- Total: $O(1)$ (constant time)

Why? Even though it runs 100 times, it doesn't depend on input size n.

Example 4: Nested Loops

```
def nested_loops(arr): # arr has n elements
    for i in range(n): # n iterations
        for j in range(n): # n iterations
            print("Hello") # O(1) operation
```

Analysis:

- Outer loop: n times
- Inner loop: n times (for each outer iteration)
- Each operation: $O(1)$
- Total: $n \times (n \times O(1)) = O(n^2)$

Visualization (n = 3):

```
i=0: 3 print statements
i=1: 3 print statements
i=2: 3 print statements
Total: 9 = 32
```

Example 5: Program with Multiple Sections

```
print("Hello") # Section 1:  $O(1)$ 

for i in range(n): # Section 2:  $O(n)$ 
    print(i)

arr.sort() # Section 3:  $O(n \log n)$ 
```

Analysis:

- Section 1: $O(1)$
- Section 2: $O(n)$
- Section 3: $O(n \log n)$

Total: $O(1) + O(n) + O(n \log n)$

- Mathematically: $1 + n + n \log n$
- Highest power term: **$n \log n$**
- Total Complexity: **$O(n \log n)$**

Why $n \log n$ dominates?

- When $n = 10^5$: $n \log n \approx 1.7 \times 10^6$ (much larger than just $n = 10^5$)

Part 7: Critical Distinction - $O(1)$ vs Constant

Common Misconception:

" $O(1)$ means the operation takes exactly 1 unit of time"

Truth:

" $O(1)$ means the operation takes constant time that doesn't change with input size"

Example:

```
def operation(arr):  
    print("Hello")    # Takes 3 seconds (constant)  
                     # This is still  $O(1)$ !
```

Whether:

- $n = 1 \rightarrow 3$ seconds
- $n = 100 \rightarrow 3$ seconds
- $n = 1,000,000 \rightarrow 3$ seconds

The time is constant $\rightarrow O(1)$

The actual numerical value (3 seconds vs 1 millisecond) doesn't matter. What matters is that it doesn't change with input size.

Part 8: Time Complexity Comparison Table

Complexity	Input $n=10$	Input $n=100$	Input $n=1000$	When to Use	Verdict
------------	-----------------	------------------	-------------------	-------------	---------

Complexity	Input n=10	Input n=100	Input n=1000	When to Use	Verdict
$O(1)$	1 op	1 op	1 op	Direct array access	☐ Always OK
$O(\log n)$	~3 ops	~6 ops	~10 ops	Binary search	☐ Almost always
$O(n)$	10 ops	100 ops	1000 ops	Simple loop	☐ Usually OK
$O(n \log n)$	~33 ops	~664 ops	~10k ops	Merge sort, Quick sort	☐ Generally OK
$O(n^2)$	100 ops	10k ops	1M ops	Nested loops (bubble sort)	☒ Limited
$O(n^3)$	1k ops	1M ops	1B ops	Triple nested loop	☐ Usually too slow
$O(2^n)$	1k ops	10^{30} ops	-	Recursive problems	☐ Only $n \leq 20$
$O(n!)$	3.6M ops	-	-	Permutations	☐ Only $n \leq 10$

Part 9: Practical Decision Making

When You See Input Size, You Know What Algorithm to Use

If $n \leq 10$: Any complexity works

- $O(2^n)$, $O(n!)$, $O(n^n)$ are all acceptable

If $n \leq 1,000$: Use up to $O(n^3)$

- $O(n^3) = 10^9$ operations (slightly over 10^8 limit but sometimes acceptable)

If $n \leq 10^5$: Use $O(n \log n)$ or $O(n)$

- $O(n^2) = 10^{10}$ operations (too slow)
- $O(n \log n) = \sim 1.7 \times 10^6$ operations (perfect)

If $n \leq 10^8$: Use only $O(n)$ or $O(\log n)$

- $O(n) = 10^8$ operations (exactly at limit)
- $O(n \log n) = 2.7 \times 10^9$ operations (too slow)

If $n \leq 10^9$: Use only $O(\log n)$

- $O(n) = 10^9$ operations (exceeds limit)
- $O(\log n) = \sim 30$ operations (excellent)

Part 10: Common Sorting Algorithm Complexities

Important: Some common algorithms have fixed complexities you should memorize:

Sorting Algorithms

- **Bubble Sort:** $O(n^2)$
- **Selection Sort:** $O(n^2)$
- **Insertion Sort:** $O(n^2)$
- **Merge Sort:** $O(n \log n)$ - efficient
- **Quick Sort:** $O(n \log n)$ average, $O(n^2)$ worst case
- **Heap Sort:** $O(n \log n)$

Key Point: Merge Sort always has $O(n \log n)$, which is why it's so important.

Part 11: Deriving Time Complexity - The Algorithm

Step-by-Step Process:

1. **Identify all independent sections** in your code
2. **For each section, identify:**
 - What is the variable that affects execution time? (usually n)
 - How many times do statements execute?
3. **Express each section's complexity**
4. **Combine all sections:**
 - Take the maximum complexity
 - Keep only the highest power of n
5. **Write final complexity**

Example Walkthrough:

```
def analyze_me(arr): # arr has n elements
    x = 5             # O(1)
    for i in range(n): # Runs n times
        print(i)      # O(1) per iteration
    sort(arr)          # O(n log n)
```

Step 1: Identify sections

- Section A: `x = 5` ($O(1)$)
- Section B: `for` loop (depends on n)
- Section C: `sort(arr)` ($O(n \log n)$)

Step 2: Analyze each

- Section A: $O(1)$
- Section B: $n \times O(1) = O(n)$
- Section C: $O(n \log n)$

Step 3: Express total

- Total = $O(1) + O(n) + O(n \log n)$
- Mathematical: $1 + n + n \log n$

Step 4: Find highest power

- Highest power term: $n \log n$
- This dominates the expression for large n

Step 5: Final answer

- **Complexity:** $O(n \log n)$

Part 12: Practice Tips

To Verify Your Answer:

If confused about which complexity is larger, **substitute actual values:**

Compare $O(n)$ vs $O(n^2)$:

- $n = 2$: $O(n) = 2$, $O(n^2) = 4 \rightarrow n^2$ is larger
- $n = 10$: $O(n) = 10$, $O(n^2) = 100 \rightarrow n^2$ is much larger

Compare $O(n^2)$ vs $O(n \log n)$:

- $n = 10$: $O(n \log n) = 10 \times 3.3 \approx 33$, $O(n^2) = 100 \rightarrow n^2$ is larger
- $n = 100$: $O(n \log n) = 100 \times 6.6 \approx 660$, $O(n^2) = 10,000 \rightarrow n^2$ is much larger

Compare $O(n)$ vs $O(n \log n)$:

- $n = 10^5$: $O(n) = 10^5$, $O(n \log n) \approx 1.7 \times 10^6 \rightarrow n \log n$ is larger

Never Memorize - Always Understand:

- Don't just memorize that "Merge Sort is $O(n \log n)$ "
- Understand WHY it's $O(n \log n)$ through analysis
- This understanding transfers to analyzing any algorithm

Summary of Key Takeaways

1. Simple Operations Take $O(1)$ Time

Any basic operation (assignment, arithmetic, print) takes constant time.

2. The 10^8 Rule is Your Guide

Online judges typically allow $\sim 10^8$ simple operations. Use this to choose your algorithm based on input size.

3. Only Highest Power Matters

When you have multiple terms, keep only the highest power of n .

4. Input Size Determines Algorithm Choice

- $n = 10$: Can use any algorithm
- $n = 1,000$: Use $O(n)$ to $O(n^2)$
- $n = 10^5$: Use $O(n)$ or $O(n \log n)$
- $n = 10^8$: Use $O(n)$ only
- $n = 10^9$: Use $O(\log n)$ only

5. Independent Sections - Take Maximum

When combining multiple sections, take the one with highest complexity.

6. Nested Loops = Multiplication

- 1 loop of n : $O(n)$
- 2 nested loops of n : $O(n^2)$
- 3 nested loops of n : $O(n^3)$

7. $O(1) \neq$ "Takes 1 Unit of Time"

$O(1)$ means constant time that doesn't change with input size.

8. When in Doubt, Substitute Values

Use actual numbers to compare which complexity is larger.

Final Thoughts

Understanding time complexity deeply gives you a superpower in problem-solving:

- ☐ You'll know immediately which algorithms will work for given input sizes
- ☐ You'll understand why interviewers ask about time complexity
- ☐ You'll be able to optimize your solutions effectively
- ☐ You'll impress interviewers with deep knowledge

This knowledge is rare. Most people memorize that "Binary Search is $O(\log n)$ " without understanding why. You now understand not just WHAT the complexity is, but WHY it is that way.

Your competitive programming and interview journey just got a major upgrade.

Quick Reference Guide

Complexity Ranking (Best to Worst)

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

Which Complexity for Which Input Size?

$n \leq 10$	$\rightarrow O(2^n), O(n!)$
$n \leq 100$	$\rightarrow O(n^3)$
$n \leq 1,000$	$\rightarrow O(n^2)$
$n \leq 10^5$	$\rightarrow O(n \log n)$
$n \leq 10^6$	$\rightarrow O(n)$
$n \leq 10^8$	$\rightarrow O(n) \text{ or } O(\log n)$
$n \leq 10^9$	$\rightarrow O(\log n)$

How to Analyze Any Code

1. Identify independent sections
2. For each section, count how many times statements execute
3. Express as function of n
4. Take maximum among all sections
5. Keep only highest power of n
6. That's your time complexity!

Created from: "What If You Could Master Time Complexity in Just 30 Minutes?" by Padho with Pratyush (Oct 20, 2025)

Remember: Time Complexity is not just theory—it's your practical tool for writing efficient code that passes interviews and competitive programming contests.